Range-Certified Dot-Product Tiling for Small-Field GPU Proofs

Lee Si Yuan
Independent Researcher
siyuanlee.cs@gmail.com

Abstract

Small-field proofs of integer inference must ensure that a field equation cannot hide a nonzero integer residual through modular wraparound. Sable gives gadget-specific range bounds for this purpose. We study a complementary kernel-planning extension: use verifier-enforced, output-row-specific operand intervals to calculate each dot-product tile's largest safe reduction length, subject to both the field residual bound and signed-int32 accumulation. The planner uses interval products and checks the entire residual interval, including output values. A Lean 4 development proves that these endpoint checks imply integer equality whenever the field relation holds. In a seeded synthetic workload of 512 output rows and 100,000 reduction terms, per-row bounds reduce the counted row-chunk total from 1,024 to 695 (32.1%); 329 rows need no split. This is a range-planning result, not a GPU speedup: the channel intervals are synthetic, must be enforced by the real circuit, and no GPU or proof implementation was available.

1 Introduction

Verifiable inference can prove that a committed integer model produced a claimed output, but the field equations in a proof system are modular. A constraint that is zero modulo a prime need not be zero over the integers unless its residual is bounded. Sable addresses this with gadget-specific bounds for its fixed integer transformer (Solberg, 2026). Its appendix, for example, bounds a 2^{15}-term linear-layer accumulator below 2^{29} over the KoalaBear field.

This work studies a range-planning extension for dot-product kernels. A single conservative operand bound is simple, but may force every output row to use the same reduction tile length. If the proof already enforces tighter row-specific weight intervals, interval arithmetic can certify longer tiles for some rows while retaining the exact same soundness condition. Our contribution is a small planner and a Lean-checked implication from endpoint bounds to integer equality. We do not implement a cryptographic prover or GPU kernel.

The source identifier arXiv:2611.14083v1 is absent from prior paper artifacts and paper-specific commits in this repository. Its first repository appearance is the 2026-11-11 automated daily feed. That feed includes metadata and an abstract; the full arXiv v1 text was consulted for this work. This audit is limited to repository history and available session context.

2 Interval certificates

For each term i, let activation $x_i \in [x_i^-, x_i^+]$ and weight $w_i \in [w_i^-, w_i^+]$. Define

$$\ell_i = \min\{x_i^- w_i^-, x_i^- w_i^+, x_i^+ w_i^-, x_i^+ w_i^+\},$$
$$u_i = \max\{x_i^- w_i^-, x_i^- w_i^+, x_i^+ w_i^-, x_i^+ w_i^+\}.$$

For an n-term tile, summing these bounds (and any bias interval) encloses the integer accumulator s in $[S^-, S^+]$. Independently constrain the output witness $z \in [Z^-, Z^+]$. Then

$$S^- - Z^+ \le s - z \le S^+ - Z^-. \tag{1}$$

A sufficient no-wrap certificate over a prime field of characteristic p is

$$-p < S^- - Z^+, \qquad S^+ - Z^- < p. \tag{2}$$

If the circuit proves $s - z = pk$ for integer k, Equation 2 gives $-p < s - z < p$, so $k = 0$ and $s = z$ over the integers. The formal file proves this exact implication, including the general asymmetric interval case.

The largest usable tile is the greatest n for which Equation 2 holds and the accumulator interval fits signed int32. These bounds are sound only

Metric	Result
Global safe tile length	66,052 terms
Per-row tile min / median / max	66,052 / 100,000 / 100,000
Rows fitting in one tile	329 / 512
Baseline row-chunk count	1,024
Interval-aware row-chunk count	695
Row-chunk count reduction	32.1%
Concrete dot-product terms checked	4,096

Table 1: Seeded synthetic planning results.

when the circuit enforces operand and output intervals. Per-row empirical extrema are not certificates unless they are made part of the statement and checked against the committed weights.

3 CPU range-planner experiment

The standard-library script uses seed 20261112, the KoalaBear prime $p = 2^{31} - 2^{24} + 1$, activation range $[-127, 127]$, and 512 synthetic output rows. Each row is assigned a declared symmetric weight interval $[-m, m]$, where m is drawn uniformly from $\{8, \ldots, 127\}$. It plans a reduction of 100,000 terms. The baseline uses the global weight interval for every row; the proposed planner uses each row's interval. The metric is the number of row chunks implied by the safe tile sizes, not measured GPU or proving time.

All script assertions pass, including interval corner multiplication, signed-int32 bounds, and rejection of the first tile length violating the strict field boundary. The result demonstrates a reduction in this synthetic planner's chunk count. It does not imply fewer GPU instructions: irregular row lengths can require masks and may harm occupancy or increase padding.

4 Formal verification and limitations

`RangeCertifiedDotClaims.lean` proves: (1) a multiple of positive p strictly between $-p$ and p is zero; (2) interval endpoints imply the residual lies strictly within that range; and (3) those facts imply integer equality from a modular equality. Lean checks the algebra, but not the circuit implementation, range metadata binding, or accelerator arithmetic.

The environment has no NVIDIA GPU or CUDA toolchain. No Sable prover, model, checkpoint, or GPU kernel was run. The synthetic row intervals do not establish the ranges of real quantized weights. A next experiment should bind per-row checkpoint ranges to proof constraints and measure prover time, proof size, occupancy, and correctness against a uniform-bound kernel on the same GPU.

Reproducibility

Run `python3 experiment.py` in this paper directory; it writes `results.json`. From `papers/formal`, run:

```
lake build Mathlib.Tactic.Linarith
lake env lean
RangeCertifiedDotClaims.lean
```

References

Tomer Solberg. 2026. Sable: A Small-Field Sumcheck Argument for Bit-Exact Inference of Large Language Models. arXiv:2611.14083v1. https://arxiv.org/abs/2611.14083v1.